Fase 2 do CRISP-DM

Análise Exploratória de Dados (O Jeito Certo)

Evitando Vazamento de Dados e Dominando Variáveis Categóricas

Centro de Informática / UFPE

Mineração de Dados Aplicada

Presented by

Leandro Almeida

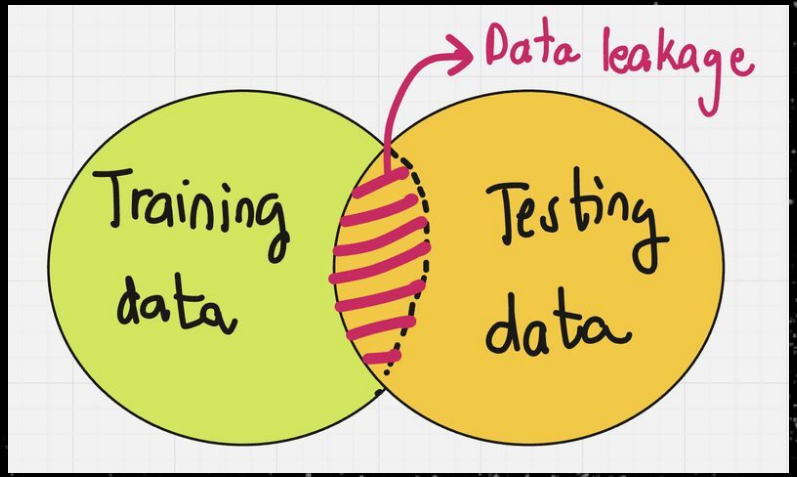
A Regra de Ouro da Ciência de Dados

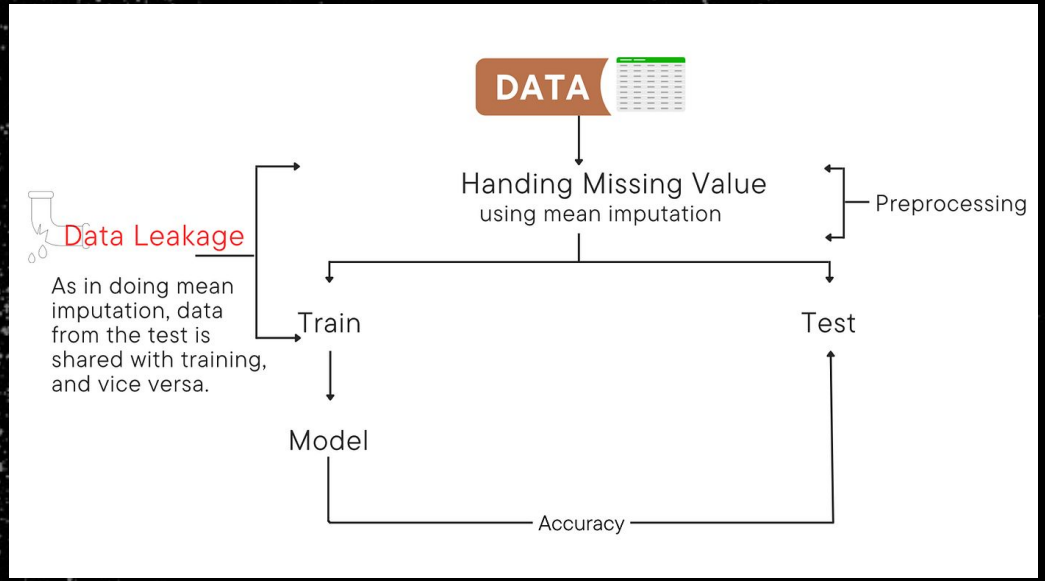
A Regra de Ouro da Ciência de Dados

NUNCA use sua base de teste para exploração, análise ou pré-processamento.

O Perigo: Vazamento de Dados (Data Leakage)

Modelos falsamente otimistas que falham no mundo real

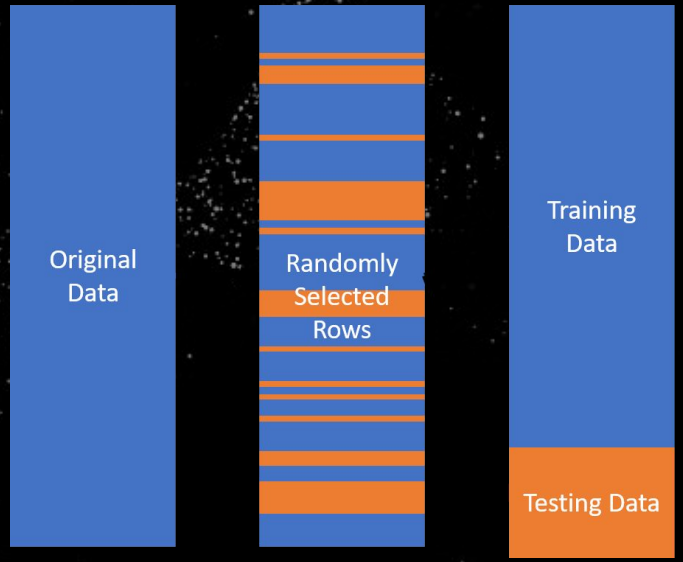




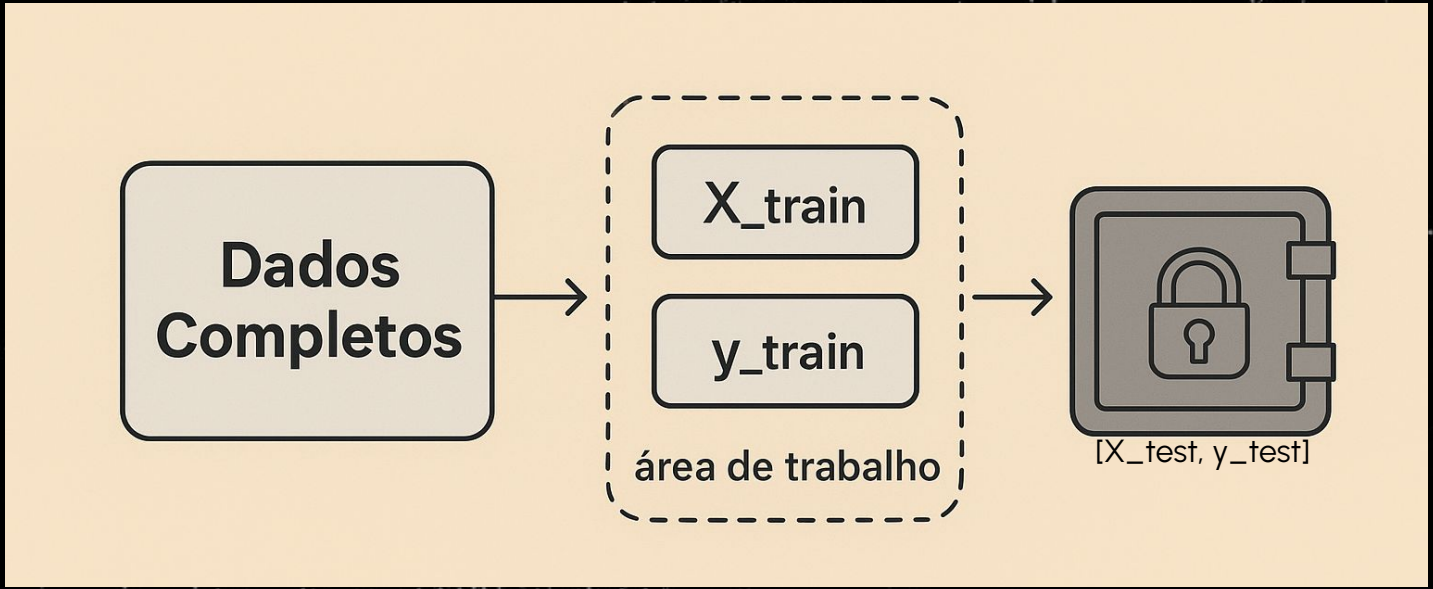
A solução

Uso adequado de funções estabelecidas para divisão dos dados brutos:

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,        # 20% para teste
    random_state=42,      # Garante reprodutibilidade
    stratify=y            # Mantém a proporção do alvo (essencial!)
)
```



Nosso foco: A base de treino



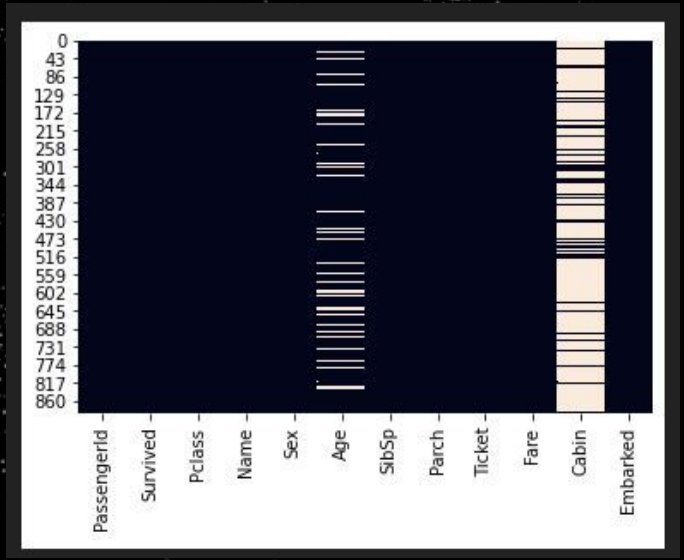
Investigando Dados Ausentes (NaN)

- Identificando e quantificando os NaNs

```
# Contar valores nulos por coluna
print("Valores nulos por coluna (contagem):")
print(X_train.isnull().sum())

print("\nValores nulos por coluna (percentual):")
print((X_train.isnull().sum() / len(X_train)) * 100)
```

- Visualizando Padrões de Ausência com Heatmap

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
sns.heatmap(X_train.isnull(), cbar=False, cmap='viridis')
plt.title('Mapa de Calor de Valores Ausentes em X_train')
plt.xlabel('Colunas')
plt.ylabel('Registros')
plt.show()
```



Estratégias Iniciais para NaNs

Remover Dados Ausentes

A remoção é a estratégia mais simples, mas deve ser usada com cautela.

- **`dropna()`:** Remove linhas ou colunas inteiras que contêm valores NaN.
  - `axis=0` (padrão): Remove linhas.
  - `axis=1`: Remove colunas.
  - `how='any'` (padrão): Remove se *qualquer* NaN estiver presente.
  - `how='all'`: Remove se *todos* os valores forem NaN.
  - `subset`: Especifica um subconjunto de colunas para verificar NaNs.

```
# Exemplo 1: Remover linhas com qualquer NaN (se o percentual for muito baixo e não impactar muito o dataset)
X_train_sem_nan_linhas = X_train.dropna(axis=0, how='any')
print(f"Shape original: {X_train.shape}")
print(f"Shape após remover linhas com NaN: {X_train_sem_nan_linhas.shape}")

# Exemplo 2: Remover uma coluna específica se tiver muitos NaNs (ex: > 70% de NaNs)
# Supondo que 'Feature_D' tenha muitos NaNs (como no exemplo anterior)
# Mas só faríamos isso se a análise mostrar que a coluna é irrelevante ou tem NaNs demais.
# X_train_sem_coluna_D = X_train.drop(columns=['Feature_D'])
# print(f"Shape após remover coluna 'Feature_D': {X_train_sem_coluna_D.shape}")
```

Estratégias Iniciais para NaNs

Preencher (Imputar) Dados Ausentes com `fillna()`

A imputação substitui os valores NaN por um valor estimado ou calculado. Esta é geralmente a abordagem preferida para não perder informações.

- **Para Variáveis Numéricas:**
  - **Média (`.mean()`):** Bom para distribuições mais simétricas.
  - **Mediana (`.median()`):** Mais robusta a outliers, ideal para distribuições assimétricas.

Original

One	Two
0	2
1	3
NaN	0
2	1

data.fillna(0)

One	Two
0	2
1	3
0	0
2	1

data.fillna(method = 'bfill')

One	Two
0	2
1	3
2	0
2	1

data.fillna(method = 'ffill')

One	Two
0	2
1	3
1	0
2	1

```
# Exemplo: Preencher NaNs em 'Feature_B' (numérica) com a mediana
mediana_feature_B = X_train['Feature_B'].median()
X_train_imputado_B = X_train.copy() # Criar uma cópia para não alterar o original diretamente
X_train_imputado_B['Feature_B'].fillna(mediana_feature_B, inplace=True)

print(f"NaNs em 'Feature_B' antes: {X_train['Feature_B'].isnull().sum()}")
print(f"NaNs em 'Feature_B' depois: {X_train_imputado_B['Feature_B'].isnull().sum()}")
```

Estratégias Iniciais para NaNs

Para Variáveis Categóricas:

- **Moda (`.mode()[0]`):** Preenche com a categoria mais frequente.
- **Valor Constante:** Preenche com uma nova categoria como 'Ausente' ou 'Desconhecido'.

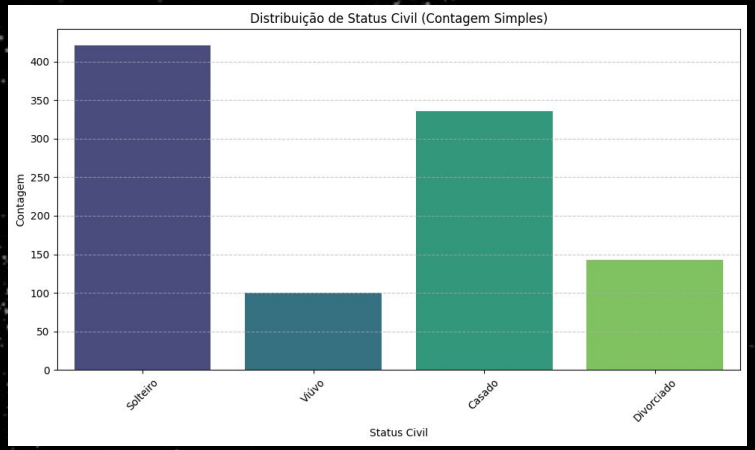
```
# Exemplo: Preencher NaNs em 'Feature_Categorica' com a moda
# Supondo que 'Feature_E' seja categórica para este exemplo
moda_feature_E = X_train['Feature_E'].mode()[0]
X_train_imputado_E = X_train.copy()
X_train_imputado_E['Feature_E'].fillna(moda_feature_E, inplace=True)

print(f"NaNs em 'Feature_E' antes: {X_train['Feature_E'].isnull().sum()}")
print(f"NaNs em 'Feature_E' depois: {X_train_imputado_E['Feature_E'].isnull().sum()}")

# Exemplo: Preencher com um valor constante 'Desconhecido'
# X_train_imputado_E_constante = X_train.copy()
# X_train_imputado_E_constante['Feature_E'].fillna('Desconhecido', inplace=True)
```

A escolha da estratégia é baseada na EDA e no contexto do negócio. Não existe uma solução única. Analise a quantidade de NaNs, o tipo da coluna, a distribuição dos dados e o impacto que cada estratégia pode ter nos seus objetivos de negócio.

Análise Categórica: Contagem Simples

A função `countplot` da biblioteca `seaborn` é ideal para isso:

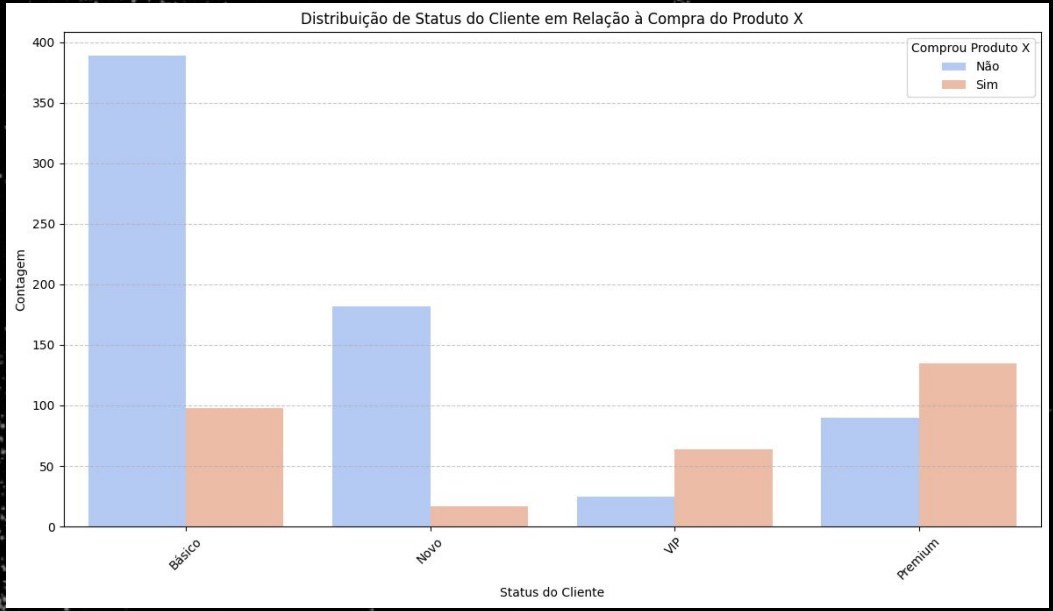
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Criação de um DataFrame de exemplo para demonstração (substitua pelo seu X_train real)
np.random.seed(42)
dados_exemplo = {
    'status_civil': np.random.choice(['Solteiro', 'Casado', 'Divorciado', 'Viúvo'], size=1000, p=[0.4, 0.35, 0.15, 0.10]),
    'idade': np.random.randint(18, 70, size=1000),
    'renda': np.random.normal(5000, 1500, size=1000)
}
X_train_exemplo = pd.DataFrame(dados_exemplo)

# Exemplo de uso do countplot
plt.figure(figsize=(10, 6))
sns.countplot(data=X_train_exemplo, x='status_civil', palette='viridis')
plt.title('Distribuição de Status Civil (Contagem Simples)')
plt.xlabel('Status Civil')
plt.ylabel('Contagem')
plt.xticks(rotation=45) # Rotaciona os rótulos do eixo X para melhor leitura
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout() # Ajusta o layout para evitar sobreposição
plt.show()
```



Categoria vs. Variável Alvo (O Gráfico-Chave)

Usamos novamente o countplot do seaborn, mas adicionamos o parâmetro hue

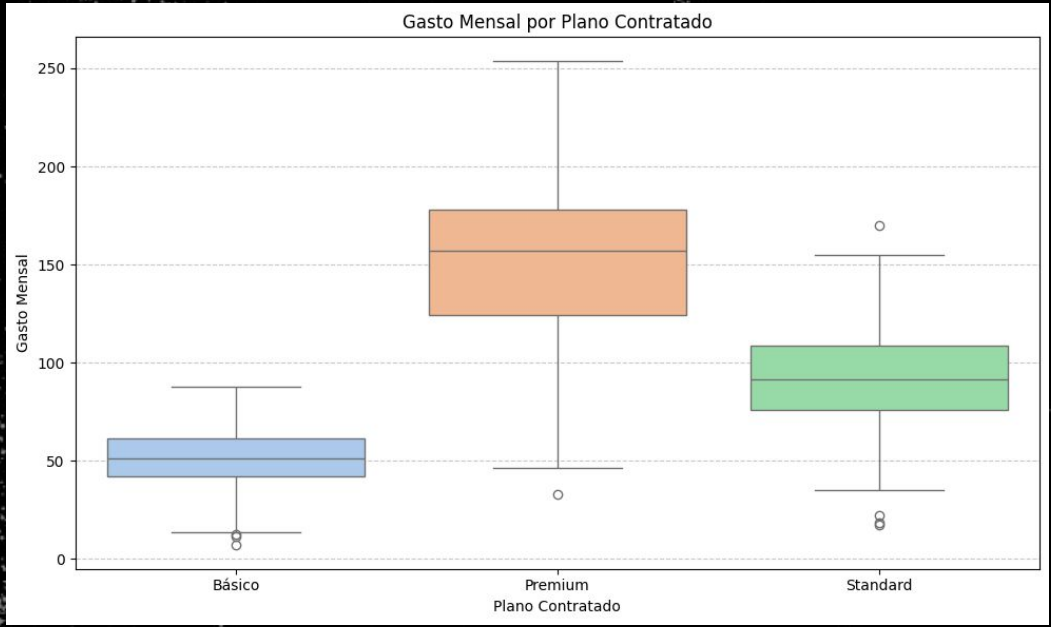
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# --- Criação de um DataFrame de exemplo para demonstração ---
# (Substitua pelo seu X_train e y_train reais)
np.random.seed(42)
n_samples = 1000

# Variável categórica 'status_cliente'
status_cliente_choices = ['Premium', 'Básico', 'VIP', 'Novo']
status_cliente_data = np.random.choice(status_cliente_choices, size=n_samples, p=[0.2, 0.5, 0.1, 0.2])

# Variável alvo binária 'compra_produto_x' (0=Não, 1=Sim)
# Vamos fazer com que 'Premium' e 'VIP' tenham mais propensão a comprar o produto X
y_data = np.zeros(n_samples, dtype=int)
for i in range(n_samples):
    if status_cliente_data[i] == 'Premium' and np.random.rand() < 0.6: # 60% de chance de comprar
        y_data[i] = 1
    elif status_cliente_data[i] == 'VIP' and np.random.rand() < 0.75: # 75% de chance de comprar
        y_data[i] = 1
    elif status_cliente_data[i] == 'Básico' and np.random.rand() < 0.2: # 20% de chance de comprar
        y_data[i] = 1
    elif status_cliente_data[i] == 'Novo' and np.random.rand() < 0.1: # 10% de chance de comprar
        y_data[i] = 1

X_train_exemplo = pd.DataFrame({'status_cliente': status_cliente_data})
y_train_exemplo = pd.Series(y_data, name='compra_produto_x') # Renomear a série para clareza

# --- Exemplo de uso do countplot com 'hue' ---
plt.figure(figsize=(12, 7))
sns.countplot(data=X_train_exemplo, x='status_cliente', hue=y_train_exemplo, palette='coolwarm')
plt.title('Distribuição de Status do Cliente em Relação à Compra do Produto X')
plt.xlabel('Status do Cliente')
plt.ylabel('Contagem')
plt.xticks(rotation=45)
plt.legend(title='Comprou Produto X', labels=['Não', 'Sim']) # Personaliza a legenda
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Categoria vs. Numérico

Podemos usar o `boxplot` do `seaborn` para isso:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# --- Criação de um DataFrame de exemplo para demonstração ---
# (Substitua pelo seu X_train real)
np.random.seed(42)
n_samples = 1000

# Variável categórica 'plano_contratado'
planos = ['Básico', 'Standard', 'Premium']
plano_data = np.random.choice(planos, size=n_samples, p=[0.4, 0.4, 0.2])

# Variável numérica 'gasto_mensal'
gasto_data = np.zeros(n_samples)
for i in range(n_samples):
    if plano_data[i] == 'Básico':
        gasto_data[i] = np.random.normal(50, 15) # Média 50, desvio 15
    elif plano_data[i] == 'Standard':
        gasto_data[i] = np.random.normal(90, 25) # Média 90, desvio 25
    elif plano_data[i] == 'Premium':
        gasto_data[i] = np.random.normal(150, 40) # Média 150, desvio 40
# Garante que gastos não sejam negativos
gasto_data = np.maximum(gasto_data, 5)

X_train_exemplo = pd.DataFrame({
    'plano_contratado': plano_data,
    'gasto_mensal': gasto_data
})

# --- Exemplo de uso do boxplot ---
plt.figure(figsize=(10, 6))
sns.boxplot(data=X_train_exemplo, x='plano_contratado', y='gasto_mensal', palette='pastel')
plt.title('Gasto Mensal por Plano Contratado')
plt.xlabel('Plano Contratado')
plt.ylabel('Gasto Mensal')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Mão na Massa - Sprints da Análise Exploratória

A Regra de Ouro:

1. ✅ **Executar o `train_test_split`** logo após carregar os dados. Garanta que `X_train`, `X_test`, `y_train` e `y_test` estão criados.
2. *Lembre-se: Tudo o que faremos a seguir é apenas com `X_train` e `y_train`.*

O Detetive de NaNs:

- ✅ **Analisar e quantificar os NaNs:** Use `.isnull().sum()` e percentuais.
- ✅ **Visualizar os NaNs:** Crie um `sns.heatmap(X_train.isnull(), cbar=False)`.
- 📝 **Anotar:** Quais colunas têm NaNs? Qual a quantidade? Há algum padrão? Qual a primeira ideia de tratamento?

Desvendando as Categorias:

- ✅ **Escolher 2-3 variáveis categóricas relevantes.**
- ✅ **Criar gráficos categóricos simples:** `sns.countplot(data=X_train, x='sua_coluna_cat')`.
- ✅ **Criar gráficos Categoria vs. Variável Alvo:** `sns.countplot(data=X_train, x='sua_coluna_cat', hue=y_train)`.
- 📝 **Anotar:** Quais categorias são mais frequentes? Quais parecem ter uma forte relação com a variável alvo?

Conectando Categorias e Números:

- ✅ **Escolher uma variável categórica e uma numérica.**
- ✅ **Criar gráficos mistos:** `sns.boxplot(data=X_train, x='sua_coluna_cat', y='sua_coluna_num')`.
- 📝 **Anotar:** Como a distribuição numérica varia entre as categorias? Há outliers significativos em alguma categoria?

Complemento da Análise Exploratória

Estudo dos materiais a seguir para enriquecer a análise exploratória dos dados:

From Data to Insights: A Beginner’s Journey in Exploratory Data Analysis
https://machinelearningmastery.com/from-data-to-insights-a-beginners-journey-in-exploratory-data-analysis/

A Data Scientist’s Essential Guide to Exploratory Data Analysis
https://medium.com/data-science/a-data-scientists-essential-guide-to-exploratory-data-analysis-25637eee0cf6

Understand Your Problem and Get Better Results Using Exploratory Data Analysis
https://machinelearningmastery.com/understand-problem-get-better-results-using-exploratory-data-analysis/

A Comprehensive Guide to Mastering Exploratory Data Analysis
https://www.dasca.org/newsroom/a-comprehensive-guide-to-mastering-exploratory-data-analysis

A Comprehensive Guide on Exploratory Data Analysis (EDA)
https://medium.com/@pabbakavya123/a-comprehensive-guide-on-exploratory-data-analysis-eda-ab38f33d6abc